



Actividad 3

Programación Funcional

Entrega

- **Lugar:** Repositorio personal de GitHub — Carpeta: Actividades/AC3
- **Fecha máxima de entrega:** 24 de septiembre a las 17:10 hrs
- **Ejecución de actividad:** La Actividad será ejecutada **únicamente** desde la terminal del computador. Los *paths* relativos utilizados en la Actividad deben ser coherentes con esta instrucción, y no pueden modificarse.

Importante: Antes de comenzar, comprueba que Git este funcionando correctamente en tu repositorio privado. Para esto, **sube los archivos base de la actividad de inmediato** (*add*, *commit*, *push*). Se espera que en esta actividad (así como en las demás actividades y tareas) utilices Git a lo largo de **todo tu desarrollo** como una herramienta, no sólo como un método de entrega. Es por esto que recomendamos enfáticamente que vayas subiendo tus cambios constantemente (*push*), ya que **problemas de último minuto** relacionados con la entrega y Git **no serán considerados**.

Objetivos de la actividad

- Implementar y usar funciones generadoras.
- Usar clases con generadores para registrar vehículos.
- Hacer consultas usando programación funcional.

Introducción

Las patentes de autos en Chile desde 2007 siguen el formato de 4 letras y 2 números (Ejemplo "DC*CC*21"). Además, existen las siguientes reglas en el formato de patentes:

- No se utilizan las vocales (A, E, I, O, U) para evitar formar palabras.
- No se utilizan las letras M, N y Ñ por su similitud entre ellas.
- La letra Q tampoco se usa por su similitud al número 0.

En esta actividad crearás un programa utilizando los conceptos de programación funcional para manejar la generación de patentes de auto y responder consultas sobre la inscripción de autos a las patentes.

Parte 1: Oficinas de patentes

En Chile hay 50 oficinas que generan patentes. Para evitar que oficinas distintas generen la misma patente para vehículos distintos, debes asignar rangos de patentes para cada oficina usando los primeros 2 caracteres de las patentes. Por ejemplo, la primera oficina recibe las patentes que comienzan con **AA** y la segunda con **AB**.

Cada oficina generará patentes en su rango específico hasta que se acaben. Una vez que se agotan las patentes del rango asignado a una oficina, se le debe asignar otro rango de patentes. No te preocupes por el caso en que se acaben todas las patentes hasta la **"ZZ-ZZ-99"** ya que se estima que esto pasará alrededor de 2029 y un alumno de este ramo ese semestre deberá resolverlo.

Crea una clase `OficinaEmisora` que permita:

- Asignar un rango único de patentes a cada instancia creada.
- Asignar un nuevo rango de patentes único cuando se acabe el anterior.
- Generar patentes usando una **función generadora** siguiendo las reglas de patentes de Chile 2007.

Tip: puede ser útil crear una función generadora de rangos de patentes y usarla para generar los rangos de cada instancia.

Parte 2: Registro de autos

Ahora que puedes generar patentes, debes crear la clase `RegistroPatentes` que maneje el registro de autos nuevos en Chile.

Para registrar un vehículo en Chile son necesarios los siguientes datos:

- Número oficina emisora: `int` entre 0 y 49
- Modelo auto: `str`
- Marca auto: `str`
- Año: `int` entre 2007 y 2026

Tu clase registro debe poder registrar un nuevo vehículo utilizando la `OficinaEmisora` indicada para generar una patente y guardar la información en una lista de diccionarios llamada `self.vehiculos`.

El formato de `self.vehiculos` debe ser:

```
1  [
2      {
3          "oficina": 1,
4          "marca": "Toyota",
5          "modelo": "Yaris",
6          "año": 2019,
7          "patente": "AB-CD-12"
8      }, {
9          "oficina": 9,
10         "marca": "Kia",
11         "modelo": "Rio",
12         "año": 2023,
13         "patente": "AJ-ZW-34"
14     }
15 ]
```

En el archivo `main.py` se te da una una función `simular_nuevo_vehiculo()` que genera vehículos randoms para registrar. Completa las secciones faltantes en el main para simular el registro de vehículos.

Parte 3: Consultas

Ahora que tienes todo el registro de vehículos funcionando, debes completar consultas sobre las patentes usando **programación funcional y funciones *lambda***.

Agrega los siguientes métodos a tu clase `RegistroPatentes` utilizando solo `map`, `filter`, `reduce` y funciones `lambda`:

- `vehiculos_por_año(año: int)`: Retorna un generador de vehículos del año dado.

Si el año dado es 2023, los datos del generador con el ejemplo anterior debería ser:

```
[{"oficina": 9, "marca": "Kia", "modelo": "Rio", "año": 2023, "patente": "AJ-ZW-34"}]
```

- `antigüedades_registros(año_actual: int)`: Retorna un generador que entregue los años de antigüedad de cada vehículo registrado.

Si el año dado es 2026, la respuesta generada para el ejemplo anterior debería ser :

```
[7, 3]
```

- `marcas_por_año()`: Retorna un diccionario donde cada llave es un año, y cada valor es un `set` con todas las marcas registradas ese año.

Si se usa con el ejemplo anterior, la respuesta sería:

```
{2019: {"Toyota"}, 2023: {"Kia"}}
```

Tip: Puedes usar `diccionario.get("key", default)` para obtener el `set` guardado en el diccionario con la llave `"key"`, pero si no existe la llave se retorna el valor por *default* dado.

Opcionalmente pueden usar el operador ternario de python.

Nota: `filter` y `map` retornan generadores, por lo que simplemente debes retornar como:

```
return filter(...)
```

```
return map(...)
```

En cambio en la función `marcas_por_año` se retorna el output de `reduce`:

```
return reduce(...)
```

Puntaje por sección

Los criterios indican los aspectos principales que serán evaluados y pueden subdividirse durante la corrección.

Criterio	Puntaje
Parte 1: Oficinas de patentes	
<code>OficinaEmisora</code> asigna rangos únicos a las oficinas y asigna un nuevo rango al agotarse el anterior.	0,75 pts
<code>OficinaEmisora</code> genera correctamente las patentes mediante una función generadora, respetando el formato y las restricciones indicadas.	0,75 pts
Parte 2: Registro de autos	
<code>RegistroPatentes</code> registra correctamente los vehículos, generando su patente mediante la <code>OficinaEmisora</code> correspondiente y manteniendo el formato de <code>self.vehiculos</code> .	0,5 pts
Parte 3: Consultas	
<code>vehiculos_por_año</code> obtiene los vehículos del año solicitado.	1 pt
<code>antiguedades_registros</code> obtiene la antigüedad de cada vehículo.	1 pt
<code>marcas_por_año</code> construye correctamente el diccionario de marcas por año.	1 pt
Descuentos: Se podrá aplicar un máximo de 1 punto de descuento por problemas de legibilidad, incumplimiento de PEP 8 o de las restricciones indicadas. Se aplicará el descuento según la siguiente escala: ▪ Hasta 3 errores: 0 puntos de descuento. ▪ Entre 4 y 7 errores: 0,5 puntos de descuento. ▪ 8 o más errores: 1 punto de descuento.	
Total	5,0 pts

Notas

- Puedes usar las librerías `collections` y `functools` pero ninguna más.
- Tu código debe poder ejecutarse por consola mediante `python main.py` o `python3 main.py`.
- Debes entregar todos los archivos de tu código dentro de la carpeta `Actividades/AC3` de tu repositorio personal.
- La entrega debe encontrarse en la rama por defecto, `main`, de tu repositorio de Git.